

[Target] VIBE ID -- AI-Powered Attendance & Class Management System

- > An intelligent biometric attendance platform leveraging Face Recognition & Voice Recognition
- > to automate classroom management -- built with Python, Streamlit, and Supabase.

Table of Contents

#	Section
1	Project Overview
2	Problem Statement
3	Key Features
4	Technology Stack
5	System Architecture
6	Database Schema
7	ML / AI Pipeline Details
8	Application Flow & User Journeys
9	Project Structure
10	Security Features
11	UI / UX Design Highlights
12	Challenges Faced & Solutions
13	Future Enhancements
14	How to Run
15	Conclusion

1 · Project Overview

VIBE ID (formerly SnapClass) is an AI-powered smart attendance system that uses Face Recognition and Voice Recognition to fully automate classroom attendance and management workflows.

Attribute	Detail
Project Name	VIBE ID
Domain	Education Technology / AI & Machine Learning
Core Language	Python 3.x

Web Framework	Streamlit
ML Libraries	dlib, scikit-learn (SVM), Resemblyzer, librosa
Database	Supabase (PostgreSQL -- Backend as a Service)
Deployment	Streamlit Community Cloud
User Roles	Teacher (password-based login) · Student (Face ID login)

How It Works -- At a Glance

- Teachers create subjects, share enrollment QR codes, and take attendance by uploading or capturing classroom photos -- the AI automatically detects faces and marks present students.
- Students log in by simply showing their face to the camera. No passwords needed.
- Voice-based attendance serves as an alternative biometric method, identifying speakers by their unique vocal signature.
- All data is persisted in a Supabase PostgreSQL cloud database with real-time access and row-level security.

2 · Problem Statement

The Problem

Traditional attendance systems in educational institutions suffer from several well-documented pain points:

- Time-consuming -- Manual roll-calls consume 5-10 minutes per session.
- [X] Error-prone -- Human recording introduces missed entries and duplicates.
- Proxy attendance -- Students can easily answer for absent peers.
- [Chart] Poor analytics -- Paper-based records are difficult to aggregate and analyze.
- [Cycle] No scalability -- Manual processes don't scale across multiple sections.

The Solution

VIBE ID eliminates all of the above by replacing manual processes with biometric AI:

Manual Process	VIBE ID Replacement
Verbal roll-call	One-click face recognition from a classroom photo
Sign-on-paper sheets	Automatic digital attendance logs in the cloud
No identity verification	128-dim face descriptor + 256-dim voice embedding
End-of-semester tallying	Real-time per-session drill-down records

No enrollment tracking	QR code-based instant enrollment
------------------------	----------------------------------

3 · Key Features

Authentication & Identity

Feature	Description
Face Recognition Login	Students authenticate by presenting their face to the camera -- no password
Dual-Role Authentication	Teachers log in via username + password; students log in via Face ID. Cle

[Camera] Attendance Automation

Feature	Description
Face Recognition Attendance	Teacher uploads or captures classroom photos; the AI detects every face
Voice Recognition Attendance	Teacher records classroom audio; the AI identifies speakers by their vocal
Multi-Face Detection	A single classroom photo can contain dozens of faces -- all are detected a

Class Management

Feature	Description
Subject Management (CRUD)	Teachers create, view, and manage subjects with section information.
Student Enrollment via QR	Students scan a QR code or enter a subject code to instantly enroll in a cla
Attendance Records	Full drill-down: summary table of all sessions -> per-session student-level

[Art] User Experience

Feature	Description
Modern Glassmorphism UI	Frosted-glass card design with subtle blur and transparency effects.
Particle Canvas Animations	Interactive HTML5 Canvas with cursor-reactive particle effects on the land
Shimmer & Pulse Animations	Gradient shimmer on titles, pulsing rings on the logo -- polished micro-inter

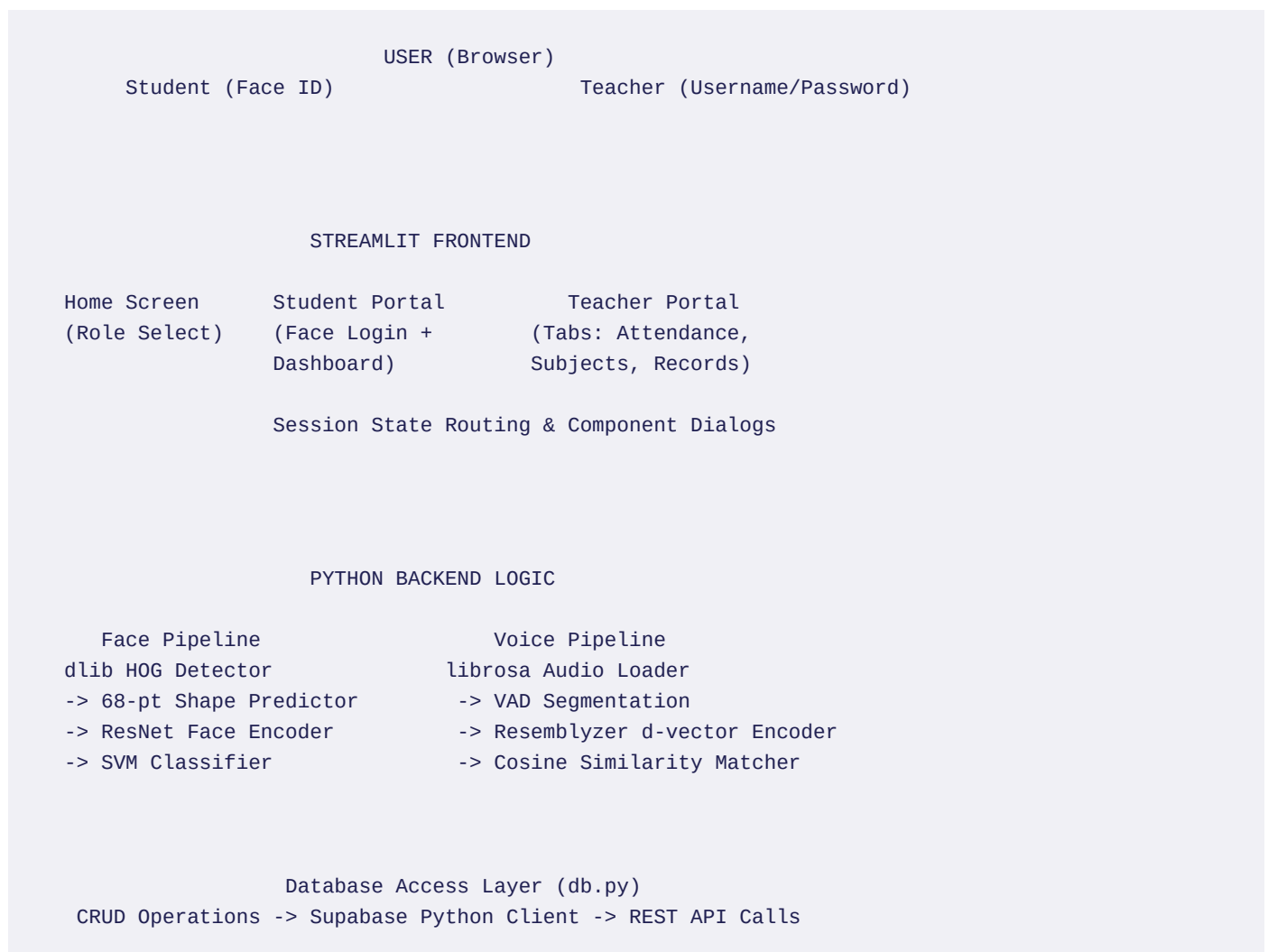
4 · Technology Stack

Technology	Purpose	Category
Python 3.x	Core programming language	Language
Streamlit	Web application framework & interactive UI	Frontend

dlib	Face detection (HOG) & 68-point facial landmarks	ML -- Vision
face_recognition_models	Pre-trained ResNet-based face recognition models	ML -- Vision
scikit-learn (SVM)	Face classification via Support Vector Machine	ML -- Classification
Resemblyzer	Voice embedding generation using the d-vector	ML -- Audio
librosa	Audio signal processing & voice activity detection	ML -- Audio
Supabase (PostgreSQL)	Cloud-hosted relational database (Backend)	Database
bcrypt	Salted password hashing for teacher accounts	Security
segno	QR Code generation for subject enrollment	Utility
Pillow (PIL)	Image loading, resizing, and preprocessing	Image Processing
NumPy	Numerical computation for embedding vectors	Data
Pandas	Data manipulation & tabular display of records	Data

5 · System Architecture

High-Level Architecture Diagram



```

SUPABASE (PostgreSQL Cloud DB)

teachers  students  subjects  subject_students  attend_logs

```

Architectural Layers

Layer	Technology	Responsibility
Presentation	Streamlit + Custom CSS/JS	UI rendering, session management, user interaction
Application	Python functions	Business logic, request handling, ML orchestration
ML Pipelines	dlib, Resemblyzer, sklearn	Biometric feature extraction & classification
Data Access	Supabase Python Client	CRUD operations via REST API
Persistence	Supabase PostgreSQL	Relational storage with row-level security

Request Flow

```

User Action -> Streamlit UI Event -> Python Handler -> ML Pipeline (if biometric)
-> Database Query/Insert (Supabase REST) -> Response Rendered in Streamlit

```

6 · Database Schema

Entity-Relationship Overview

```

teachers < subjects < subject_students > students

attendance_logs >

```

Table Definitions

teachers

Column	Type	Constraints	Description
teacher_id	UUID / INT	PRIMARY KEY	Unique teacher identifier
username	VARCHAR	UNIQUE, NOT NULL	Login username
password	VARCHAR	NOT NULL	bcrypt-hashed password
name	VARCHAR	NOT NULL	Display name

students

Column	Type	Constraints	Description
student_id	UUID / INT	PRIMARY KEY	Unique student identifier
name	VARCHAR	NOT NULL	Display name
face_embedding	FLOAT[]		128-dimensional face descriptor vector
voice_embedding	FLOAT[]	nullable	256-dimensional d-vector embedding

subjects

Column	Type	Constraints	Description
subject_id	UUID / INT	PRIMARY KEY	Unique subject identifier
subject_code	VARCHAR	UNIQUE, NOT NULL	Shareable enrollment code
name	VARCHAR	NOT NULL	Subject name
section	VARCHAR		Section identifier
teacher_id	UUID / INT	FOREIGN KEY -> teachers	Owning teacher

subject_students (Junction Table -- Many-to-Many)

Column	Type	Constraints	Description
student_id	UUID / INT	FK -> students	Enrolled student
subject_id	UUID / INT	FK -> subjects	Enrolled subject
		PK (student_id, subject_id)	Composite primary key

attendance_logs

Column	Type	Constraints	Description
student_id	UUID / INT	FK -> students	Student marked
subject_id	UUID / INT	FK -> subjects	Subject session
timestamp	TIMESTAMPTZ	NOT NULL	Date/time of the attendance session
is_present	BOOLEAN	NOT NULL	Whether the student was marked present

7 · ML/AI Pipeline Details

7.1 -- Face Recognition Pipeline

The face recognition system uses a multi-stage pipeline combining classical computer vision with deep learning:

Input Image

Stage 1: Face Detection
 dlib HOG-based frontal face
 detector (image upsampled 3×
 for small / distant faces)

Bounding boxes

Stage 2: Landmark Prediction
 68-point shape predictor
 locates eyes, nose, mouth,
 jawline for face alignment

Aligned face chips

Stage 3: Embedding Generation
 dlib's ResNet-based face
 recognition model produces a
 128-dimensional descriptor

128-D float vector

Stage 4: Classification
 SVM (linear kernel, probability
 enabled) trained on enrolled
 student embeddings
 + Euclidean distance 0.45
 threshold for strict matching

Key Technical Decisions

Decision	Rationale
HOG detector over CNN	Faster on CPU; sufficient accuracy for front-facing classroom photos.
3× upsampling	Ensures small faces in group photos are reliably detected.
SVM with linear kernel	Fast to train and predict; works well in the 128-D embedding space.
Euclidean threshold = 0.45	Stricter than the default 0.6 -- minimizes false positives in a classroom setting.
@st.cache_resource	Models (detector, predictor, encoder) cached across reruns for performance.

Registration vs. Attendance

Phase	What Happens
Registration	Student's face is captured -> embedding extracted -> stored in Supabase.
Attendance	SVM re-trained on all stored embeddings -> classroom photo scanned -> e

7.2 -- Voice Recognition Pipeline

The voice recognition system identifies speakers by their unique vocal characteristics using speaker embedding technology:

Audio Input (WAV / recorded)

Stage 1: Audio Loading

librosa loads audio at 16 kHz
mono channel

Raw waveform

Stage 2: Voice Activity

Detection (VAD)

librosa.effects.split with
30 dB threshold segments
speech from silence

Speech segments

Stage 3: Segment Filtering

Discard segments < 0.5 seconds
(too short for reliable ID)

Valid segments

Stage 4: Embedding Generation

Resemblyzer VoiceEncoder
generates 256-dimensional
d-vector for each segment

256-D float vector

Stage 5: Speaker Matching

Cosine similarity computed
against all enrolled student
voice profiles
Threshold 0.65 for match
Best match returned with
confidence score

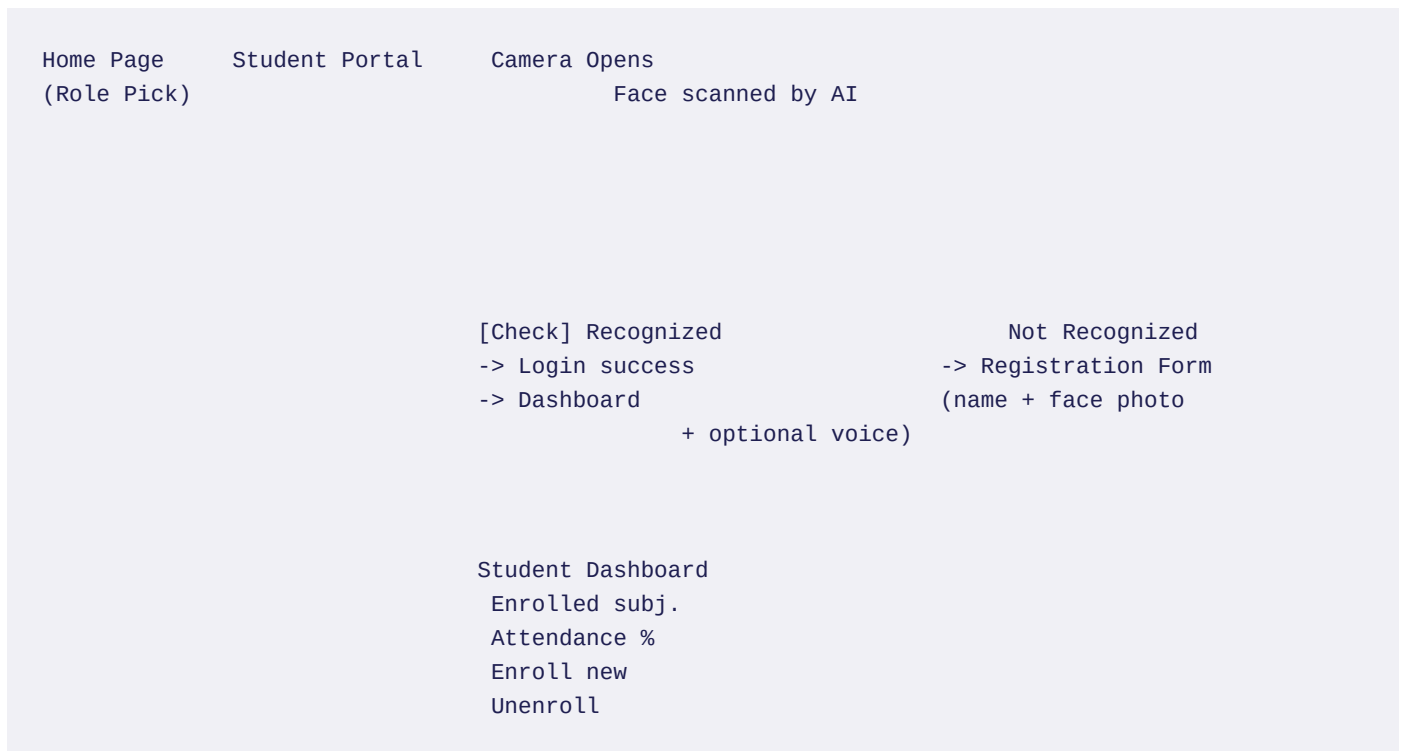
Key Technical Decisions

Decision	Rationale
16 kHz sample rate	Standard for speech processing; balances quality and processing speed.

30 dB VAD threshold	Filters background noise while retaining clear speech.
0.5 s minimum segment length	Short bursts (coughs, clicks) are unreliable for speaker ID.
Cosine similarity 0.65	Empirically tuned to balance true positive rate with false positive rate.
Resemblyzer d-vector model	Pre-trained on thousands of speakers; generalizes well without fine-tuning

8 · Application Flow & User Journeys

8.1 -- Student Journey



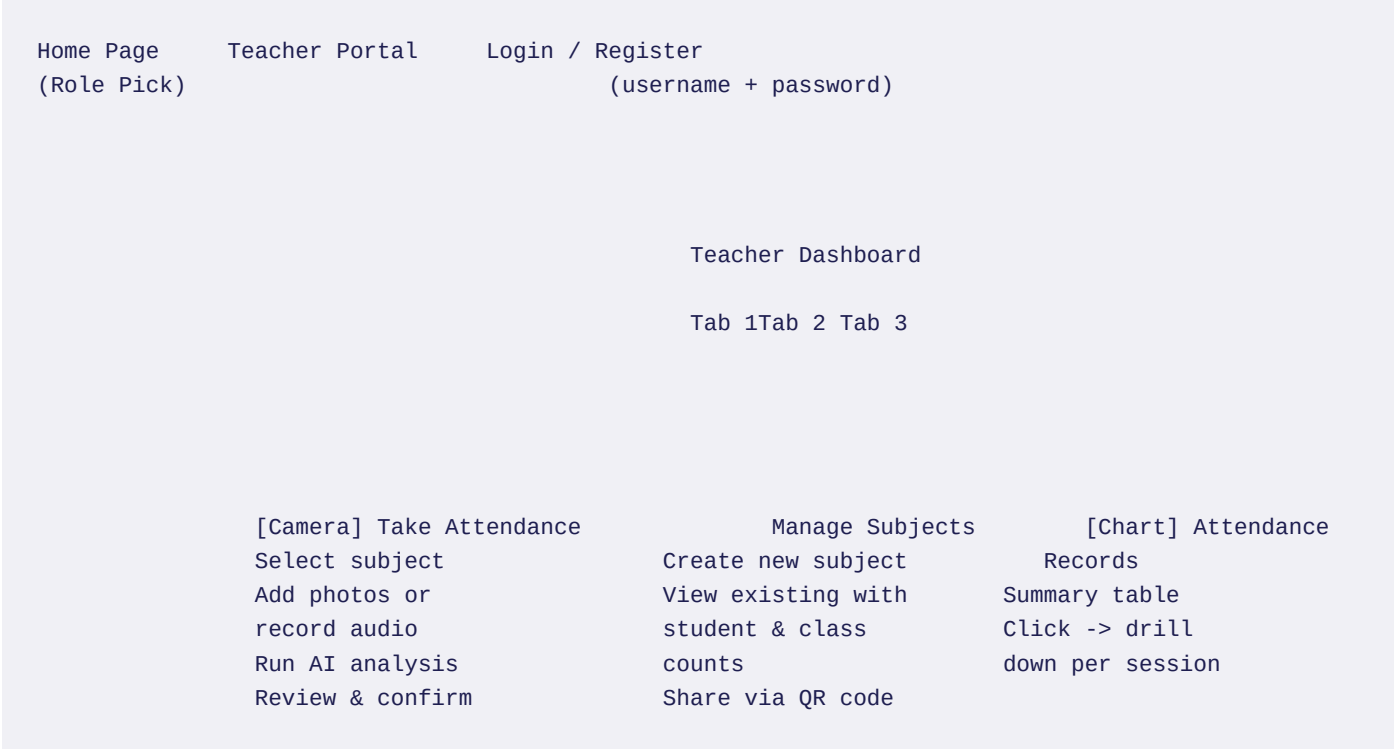
Step-by-step:

- Home -> Select "Student Portal"
- Camera opens -> Face is scanned in real-time by the AI pipeline
- If recognized -> Automatic login; student sees their personalized dashboard

with enrolled subjects and attendance statistics

- If not recognized -> Prompted with a registration form to provide name, capture a face photo, and optionally record a voice sample
- Dashboard actions -> View enrolled subjects, check attendance stats, enroll in new subjects (via code or QR), or unenroll from existing ones

8.2 -- Teacher Journey



Tab Details:

Tab	Capabilities
Take Attendance	Select a subject -> Add photos via camera or file upload -> Run Face Analysis
Manage Subjects	Create new subjects with name, code, section -> View all subjects with enrollment
Attendance Records	View a summary table of all past attendance sessions -> Click any session for details

9 · Project Structure

```
SnapClassSehaj/  
  
app.py                # [Rocket] Main entry point -- Streamlit app launcher  
Self.py              # Alternate student screen (development)  
chat_gpt.py          # Alternate student screen with cosine similarity  
Requirementss.txt    # [Package] Python dependencies list  
  
.streamlit/  
  secrets.toml        # Supabase URL & API keys (gitignored)  
  
src/  
  
  screens/            # Page-Level Screens  
    home_screen.py    # Landing page with role selection  
    student_screen.py # Student Face ID login + dashboard  
    teacher_screen.py # Teacher login + tabbed dashboard  
  
  components/         # Reusable UI Components  
    snapheader.py     # Hero header & dashboard header  
    footer.py         # Footer component
```

```
subject_card.py           # Reusable subject card widget
dialog_add_photo.py       # Photo capture / upload dialog
dialog_attendance_results.py # Review & confirm attendance dialog
dialog_auto_enroll.py     # Auto-enrollment via QR deep link
dialog_create_subject.py  # Create new subject dialog
dialog_enroll.py          # Manual subject enrollment dialog
dialog_share_subject.py   # QR code & shareable link dialog
dialog_voice_attendance.py # Voice attendance recording dialog

pipelines/               # ML / AI Pipelines
  face_pipeline.py       # Face detection, embedding, SVM classification
  voice_pipeline.py      # Voice embedding, speaker identification

database/                # Data Access Layer
  config.py              # Supabase client initialization
  db.py                  # All database CRUD operations

ui/                       # Theming & Layout
  base_layout.py         # CSS themes, animations, particle canvas JS
```

> Total modules: ~20 Python files, cleanly separated by concern.

10 · Security Features

Security Measure	Implementation Detail
[Key] Password Hashing	Teacher passwords hashed with bcrypt (salted) -- plaintext never stored.
[Shield] Row-Level Security (RLS)	Supabase RLS policies restrict data access at the database level.
No Raw Image Storage	Only the computed numeric embedding vector is stored -- not photos.
[Lock] Session-Based Auth	Authentication state managed via st.session_state -- no cookies exposed.
[Folder] Secrets Management	Supabase credentials stored in .streamlit/secrets.toml, excluded from VCS
Threshold-Based Rejection	Unrecognized faces are rejected (distance > 0.45) -- no forced matches.

11 · UI/UX Design Highlights

Visual Design Language

VIBE ID employs a dual-theme design approach:

Context	Theme
Landing Page	Dark aurora theme -- deep gradients, animated orbs, particle effects
Dashboards	Clean light theme -- white cards, clear typography, high readability

Signature UI Elements

- Animated Gradient Orbs -- Floating aurora blobs on the home page using CSS keyframe animations
- [Sparkles] Particle Canvas -- Interactive HTML5 Canvas with cursor-reactive particles, rendered via st.html() with embedded JavaScript
- Glassmorphism Cards -- Frosted-glass effect cards using backdrop-filter: blur() with semi-transparent backgrounds and subtle borders
- Shimmer Gradient Title -- The "VIBE ID" heading uses an animated gradient sweep (shimmer effect) via CSS background-clip: text
- Pulse Ring Animation -- Logo icon surrounded by expanding concentric pulse rings
- Feature Badges -- Pop-in animated badges showcasing key features on the landing page
- Two-Column Layouts -- Responsive st.columns() layouts for side-by-side content

Typography

Usage	Font Family	Style
Headings	Climate Crisis	Bold, display typeface
Body	Outfit	Clean, modern sans-serif

> Both fonts loaded from Google Fonts for consistent cross-platform rendering.

12 · Challenges Faced & Solutions

#	Challenge	Root Cause	Solution Implemented
1	SVM failing with a single class	SVM requires 2 classes to train	Added Euclidean distance threshold (0.4
2	False positives in face recognition	Default dlib threshold (0.6) was too high	Reduced threshold to 0.45 -- stricter match
3	Multiple faces in a single image	Classroom photos contain many faces	Pipeline iterates over all detected face b
4	Voice segments too short for reliable transcription	Coughs, laughs, and short sound segments	Filter out audio segments shorter than 0.5s
5	st.components.v1.html deprecated	Streamlit deprecated the old component	Migrated all custom HTML/JS rendering

13 · Future Enhancements

Priority	Enhancement	Description
High	Liveness Detection (Anti-Spoofing)	Detect printed photos or screen replays to prevent spoofing
High	Multi-Photo Face Enrollment	Capture multiple angles during registration for improved accuracy
Medium	Attendance Analytics & Charts	Visual dashboards with trend lines, heatmaps, and performance metrics
Medium	Export to CSV / Excel	Allow teachers to download attendance records in spreadsheet format
Medium	Email / SMS Notifications	Notify students or parents when attendance is marked or when a login fails
Low	Mobile-Responsive Design	Optimize layouts and interactions for smartphones and tablets
Low	Multi-Language Support	Internationalize the UI for non-English-speaking institutions

14 · How to Run

Prerequisites

- Python 3.8 or higher
- A Supabase project with the required tables created
- CMake and a C++ compiler (required for dlib installation)

Installation & Launch

```
# 1. Clone the repository
git clone <repository-url>
cd SnapClassSehaj

# 2. Install dependencies
pip install -r Requirementss.txt

# 3. Configure Supabase credentials
# Create .streamlit/secrets.toml with:
# SUPABASE_URL = "your-project-url"
# SUPABASE_KEY = "your-anon-key"

# 4. Run the application
streamlit run app.py
```

Deployment (Streamlit Cloud)

- Push the repository to GitHub.
 - Connect the repo to Streamlit Community Cloud.
 - Add SUPABASE_URL and SUPABASE_KEY as Secrets in the Streamlit Cloud dashboard.
 - Deploy -- the app will be live at <https://<your-app>.streamlit.app>.
-

15 · Conclusion

VIBE ID demonstrates the practical application of AI and Machine Learning in solving a real-world classroom problem. By combining:

- [Brain] Face Recognition -- dlib's HOG detector + ResNet encoder + SVM classifier
- Voice Recognition -- Resemblyzer's d-vector model + cosine similarity matching
- Cloud Database -- Supabase PostgreSQL with row-level security
- [Art] Modern Web UI -- Streamlit with custom glassmorphism design

the system delivers a fast, accurate, and user-friendly attendance management

solution that eliminates manual processes, prevents proxy attendance, and provides

teachers with instant, drill-down attendance records.

> *VIBE ID is not just an attendance tool -- it's a proof-of-concept for how
> biometric AI can seamlessly integrate into everyday educational workflows.*

<div align="center">

Built with using Python, Streamlit, dlib, Resemblyzer & Supabase

</div>